

PCC-253-COM: Computer Organization and Microprocessor

Unit 2: Memory Management

Characteristics of Memory System,

Q1. explain the Characteristics of Memory System.

Characteristics of Memory System

The memory system of a computer is characterized by the following important parameters.

- ① Location :- Memory can be located inside the CPU (registers, cache), main memory (RAM) or outside the CPU (secondary memory).
- ② Capacity :- It refers to the total amount of data that can be stored in the memory. It is usually measured in bytes, KB, MB, GB, etc.
- ③ Unit of Transfer :- The basic unit of data that can be read from or written into the memory at a time is called the unit of transfer. It may be 1 bit, 1 byte, word or block.
- ④ Access Method :- The way in which data is accessed.
 - Sequential Access - Data is accessed in a sequence.
 - Direct Access - Data is accessed directly to a particular location.
 - Random Access - Data can be accessed randomly in any order.
- ⑤ Performance :- Performance of memory is measured by the following parameters.
 - Access Time (T_a) - Time required to access the memory once the address is available.
 - Cycle Time (T_c) - Minimum time between two successive memory operations.
 - Transfer Rate - Number of bytes transferred per second.
- ⑥ Physical Type :- Memory can be semiconductor based (RAM, ROM) or magnetic based (Hard disk, Tape).
- ⑦ Volatility :- If the memory loses its contents when power is turned off, it is called volatile memory (e.g. RAM). If it retains the data even when power is off, it is called non-volatile memory (e.g. ROM, HDD).
- ⑧ Cost :- Cost per bit of memory. Generally, as capacity increases, cost per bit decreases.

Memory Hierarchy (Example)

Level	Memory Type	Capacity	Speed	Cost/bit
1	Registers	Very small (Bytes)	Very Fast	Very High
2	Cache Memory	Small (KB-MB)	Fast	High
3	Main Memory (RAM)	Medium (GB)	Medium	Medium
4	Secondary Memory (HDD/SSD)	Large (TB)	Slow	Low
5	Tertiary Memory (Tape, Optical)	Very Large (TB+)	Very Slow	Very Low

Summary

A memory system is characterized by its location, capacity, unit of transfer, access method, performance, physical type, volatility and cost.

The memory hierarchy

Q.1] Explain Memory Hierarchy with neat diagram?

MEMORY HIERARCHY

Memory hierarchy is an arrangement of memory systems in the order of increasing capacity and decreasing speed so that cost per bit is minimized and overall system performance is improved.

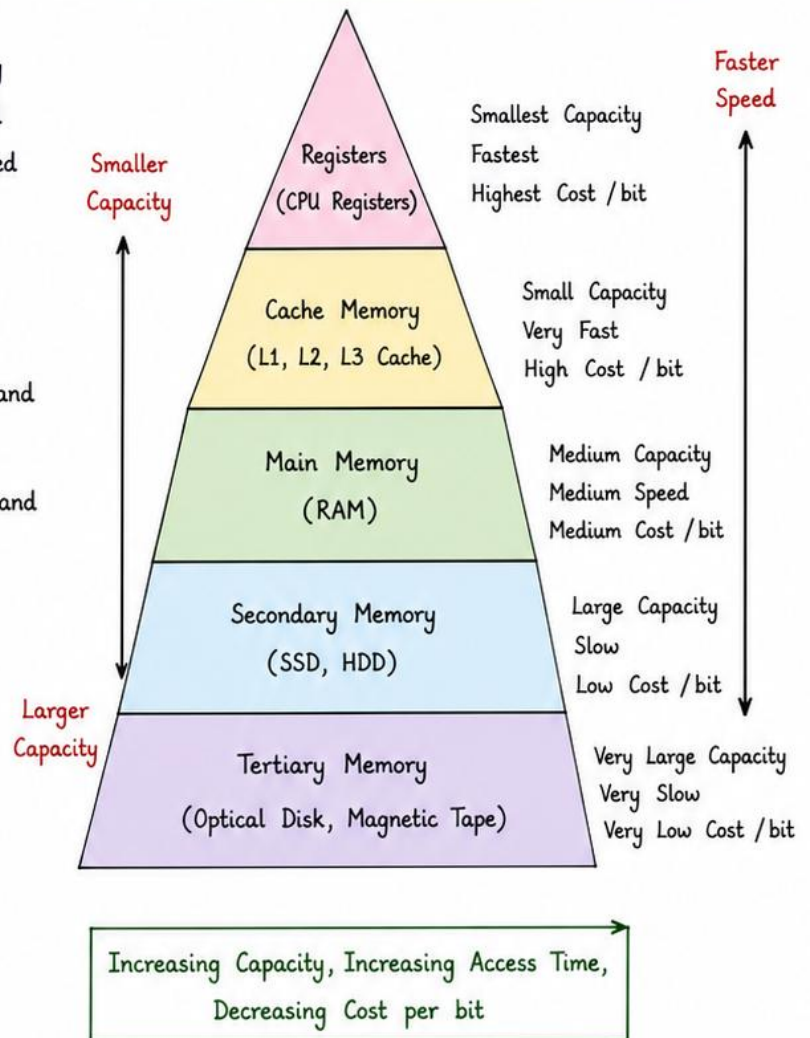
BASIC IDEA :

- Small amount of data is kept in the fastest and most expensive memory.
- Large amount of data is kept in the slowest and cheapest memory.
- Data is transferred between adjacent levels.

CHARACTERISTICS / PURPOSE :

- Provide large memory capacity.
- Provide high speed.
- Reduce average memory access time.
- Reduce cost per bit.
- Exploit locality of reference (temporal & spatial).

NEAT DIAGRAM OF MEMORY HIERARCHY



HIERARCHY STRUCTURE :

Data that is needed by CPU is brought from lower level to upper level. If data is not found at upper level (miss), it is brought from next lower level. This process continues until data is found.

KEY POINT :

Memory hierarchy improves performance by keeping frequently used data in faster memory (using locality of reference) and less frequently used data in larger, slower and cheaper memory.

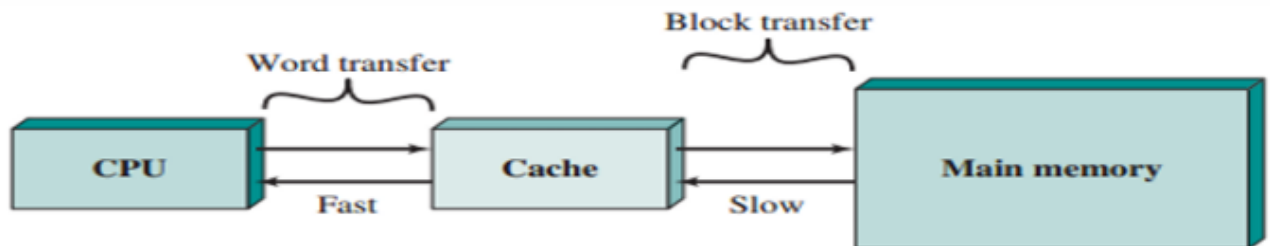
Level	Example	Capacity	Speed	Cost per bit
1	Registers	Very Small	Very Fast	Very High
2	Cache Memory	Small	Fast	High
3	Main Memory (RAM)	Medium	Medium	Medium
4	Secondary Memory (SSD, HDD)	Large	Slow	Low
5	Tertiary Memory (Optical Disk, Magnetic Tape)	Very Large	Very Slow	Very Low

Cache Memory- Cache memory principles, Elements of cache design cache address, size, and map ping functions.

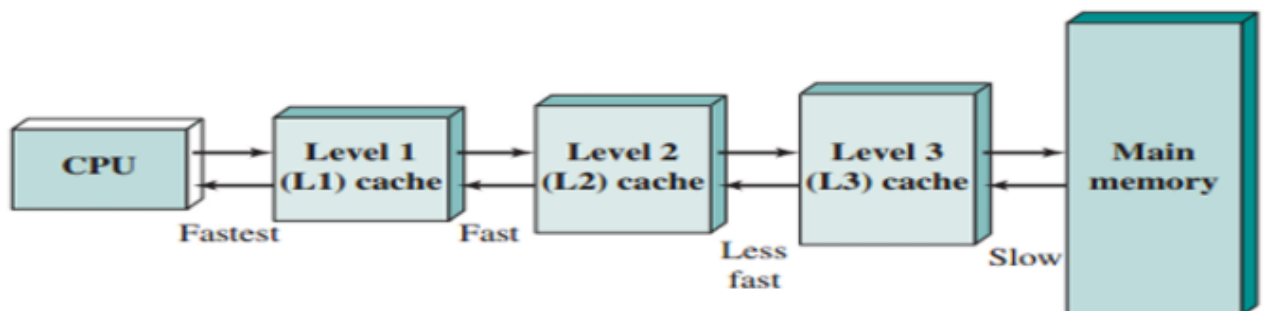
Cache memory principles

- › A cache is a high speed memory that is located between CPU and main memory and keeps a copy of the frequently used data
- › When the CPU wants a data value from memory, it first looks in the cache
- › If the data is in the cache, it uses that data
- › If the data is not in the cache, it copies a **line** of data from memory to the cache and gives the CPU what it wants
- › **Cache hit**: When the processor wants information at a given address and that information is already in the cache
- › **Cache miss**: If the processor wants information that is not already in the cache

Cache memory principles



(a) Single cache



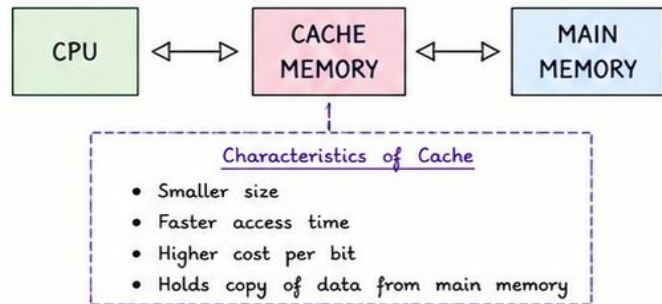
(b) Three-level cache organization

2.1 Cache Memory Principles

- **Q.2] Explain Principles of Cache Memory**
- **Q.1] Define Hit and Miss in cache memory**

2.1 CACHE MEMORY PRINCIPLES

Cache memory is a small, fast memory placed between CPU and main memory. It stores copies of the most frequently used data and instructions from main memory so that CPU can access them **faster**.



PRINCIPLES OF CACHE MEMORY

Cache memory works on the principle of **locality of reference**.

The main principles are:

1. Locality of Reference

- **Temporal Locality:** If a data item is used, it will be used again in the near future.
- **Spatial Locality:** If a data item is used, nearby data items are likely to be used soon.

2. Cache Stores Copies

Cache keeps a copy of recently/frequently used blocks from main memory. CPU first checks cache; if data is found, it is supplied immediately.

3. Automatic Transfer

When a block is not in cache, the entire block containing that data is brought from main memory to cache (principle of block transfer).

4. Transparency

CPU accesses memory without knowing whether data is in cache or in main memory. Cache operation is hardware managed.

5. Replacement

If cache is full and new block has to be loaded, an existing block is replaced according to a replacement policy (e.g., LRU, FIFO, Random).

6. Write Policy

When writing data, cache uses a write policy:

- **Write Through** - data written to both cache and main memory.
- **Write Back** - data written only to cache, main memory updated when the block is replaced.

7. Performance Improvement

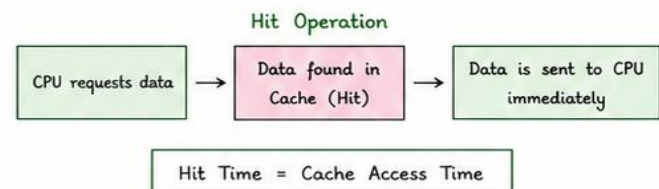
By keeping frequently used data close to CPU, cache reduces average memory access time and improves overall system performance.

HIT AND MISS IN CACHE MEMORY

1) Hit

A Hit occurs when the required data/instruction is **found in the cache**.

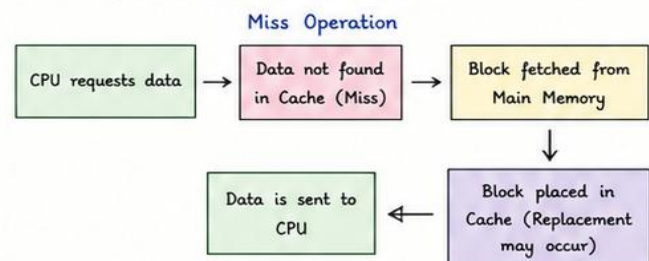
→ CPU gets the data from cache (fast access).



2) Miss

A Miss occurs when the required data/instruction is **not found in the cache**.

→ The block is fetched from main memory and placed in cache, then data is sent to CPU.



Miss Penalty = Cache Access Time + Main Memory Access Time

AMAT (Average Memory Access Time)

AMAT = (Hit Rate × Hit Time) + (Miss Rate × Miss Penalty)

DIFFERENCE BETWEEN HIT AND MISS

Parameter	Hit	Miss
Definition	Required data is found in cache.	Required data is not found in cache.
Source of Data	Cache memory	Main memory
Access Time	Very fast (Cache access time)	Slow (Cache + Main memory access time)
Block Transfer	No block transfer needed	Block is brought from main memory to cache
CPU Wait Time	Less	More
Performance Impact	Improves performance	Degrades performance (higher miss penalty)

2.2 Elements of Cache Design

Q.1] Explain elements of Cache (Cache Size, Cache Address)?

2.2 ELEMENTS OF CACHE DESIGN

The main elements of cache design are **Cache Size** and **Cache Address**.

1. CACHE SIZE

- Cache size means how much data cache can store.
- Cache is smaller than main memory but faster.

How Cache Size is Calculated

- Total Cache Size = Number of Cache Lines $\times$ Block Size
- Block Size = Number of words/bytes in one block
- Number of Cache Lines = Total Cache Size / Block Size
- Cache size affects:
 - Hit ratio : Larger cache $\rightarrow$ more hits
 - Access time : Larger cache $\rightarrow$ slightly more time
 - Cost : Larger cache $\rightarrow$ more expensive

Example :

If a cache has 64 lines and block size is 16 words (1 word = 2 bytes),

- Total Cache Size in words = $64 \times 16 = 1024$ words
- Total Cache Size in bytes = $1024 \times 2 = 2048$ bytes
= 2 KB

Structure of Cache Memory



Typical Cache Sizes

Level	Typical Size	Use
L1 Cache	16 KB – 128 KB	Inside CPU (very fast)
L2 Cache	256 KB – 2 MB	Between L1 and main memory
L3 Cache	2 MB – 64 MB	Last level cache (shared)

2. CACHE ADDRESS

- Cache address is the way to find data in cache.
 - CPU gives an address and it is divided into 3 parts.
- The 3 parts are: **Tag**, **Index (or Set)** and **Block Offset**.

Address Format



- **Tag (t bits)** : Tells which memory block is stored in that cache line.
- **Index (s bits)** : Tells which cache line (or set) to look.
- **Block Offset (b bits)** : Tells exact word/byte inside the block.

Example :

Main memory address = 20 bits
Cache has 64 lines (direct mapped)
Block size = 16 words
Find number of bits in each field.

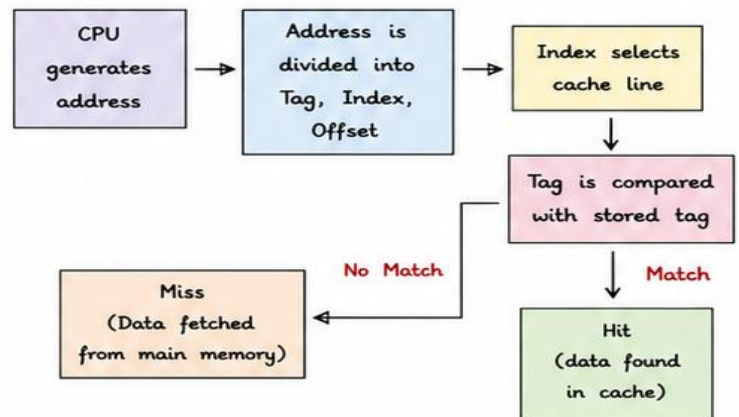
Solution :

- Block size = 16 words = $2^4 \rightarrow$ Offset bits (b) = 4
- Number of lines = 64 = $2^6 \rightarrow$ Index bits (s) = 6
- Tag bits (t) = $20 - (s + b) = 20 - (6 + 4) = 10$

Therefore,

Field	Tag	Index	Block Offset	Total
Bits	10 bits	6 bits	4 bits	20 bits

How Cache Address Works (Direct Mapping Example)



Summary of Elements

Element	Description
Cache Size	• Total storage capacity of cache. • Calculated by number of lines $\times$ block size. • Affects hit ratio, access time and cost.
Cache Address	• Method to locate data in cache. • Divided into Tag, Index (or Set), and Block Offset fields. • Used for mapping and comparison.

2.3 CACHE MAPPING TECHNIQUES

Q.3] Explain Direct Mapping with example.

DIRECT MAPPING

- In Direct Mapping, each block of main memory can go to **only one specific line** of cache.
- The block number of main memory is mapped to the cache line using:

$$\text{Cache Line Number} = (\text{Block Number}) \bmod (\text{Number of Cache Lines})$$

- Simple and fast.
- But more conflict misses may occur.

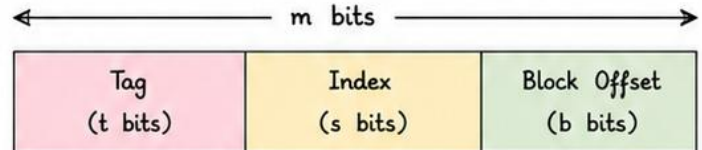
EXAMPLE

Main memory has 16 blocks (numbered 0 to 15).
Cache has 4 lines (numbered 0 to 3).
Block size = 1 word (for simplicity).

Mapping:

Main Memory Block (Block Number)	Cache Line Number (Direct Mapping)
0	$0 \bmod 4 = 0$
1	$1 \bmod 4 = 1$
2	$2 \bmod 4 = 2$
3	$3 \bmod 4 = 3$
4	$4 \bmod 4 = 0$
5	$5 \bmod 4 = 1$
:	:
15	$15 \bmod 4 = 3$

ADDRESS FORMAT (Direct Mapping)

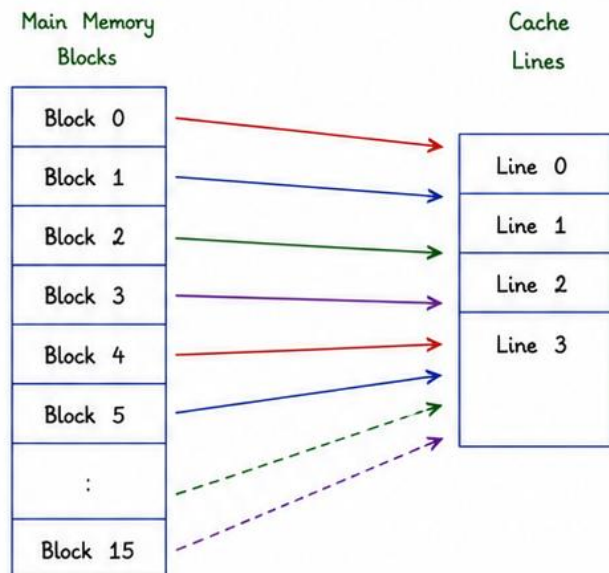


Tag : Identifies the block.

Index : Selects the cache line.

Block Offset : Selects the word/byte in the block.

Mapping Diagram



How it works?

- CPU generates an address and it is divided into Tag, Index and Block Offset.
- Index bits select the cache line.
- The Tag stored in that line is compared with the incoming Tag.
- If both match → Hit (data found in cache).
- If not match → Miss (data fetched from main memory and placed in that line).

Advantages

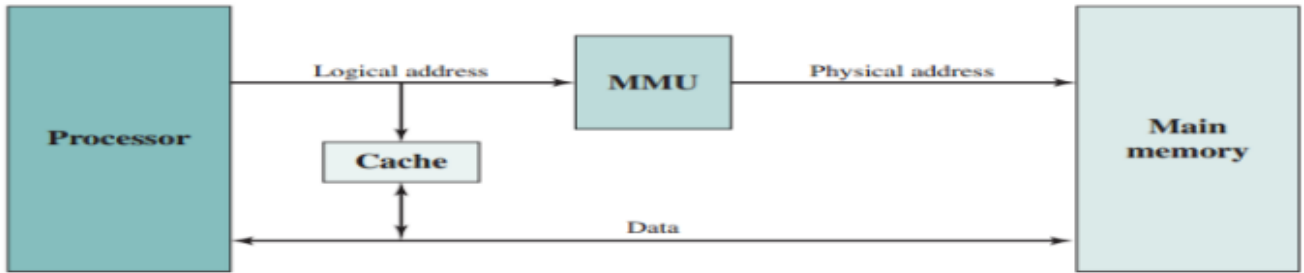
- Simple hardware
- Fast access

Disadvantages

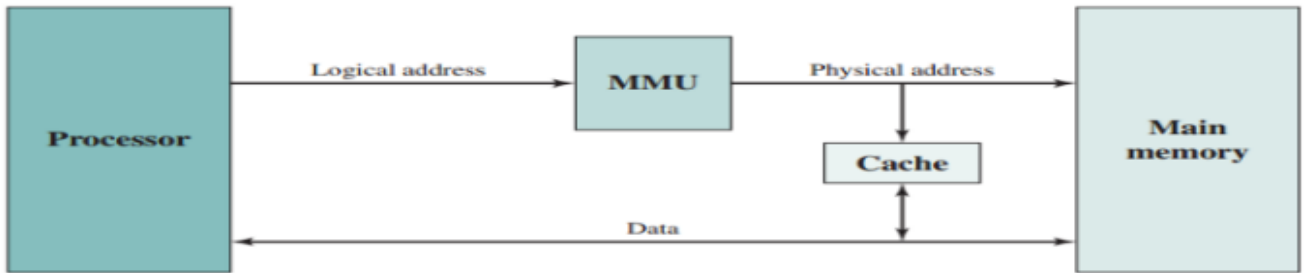
- More conflict misses
- Less flexible

Conclusion: Direct mapping is the simplest mapping technique where each main memory block can be placed in only one specific cache line.

Cache addresses



(a) Logical cache



(b) Physical cache

Q.5] Explain Fully Associative Mapping with Example.

Answer :

1) Definition :

In Fully Associative Mapping, any memory block can be placed in any line (or block) of the cache. There is **no fixed location** for a block.

Hence, a block can be loaded in any cache line.

2) Address Format :

Tag	Word/Byte Offset
(m-w) bits	w bits

- There is no Index field.

3) Working :

- When a memory block is to be accessed, its Tag is compared with the Tag of all cache lines simultaneously.
- If a match is found, it is a Hit.
- If no match is found, it is a Miss and the block is loaded in any one of the cache lines (selected by a replacement policy).

5) Advantages :

- Least conflict misses.
- Most flexible.

4) Example :

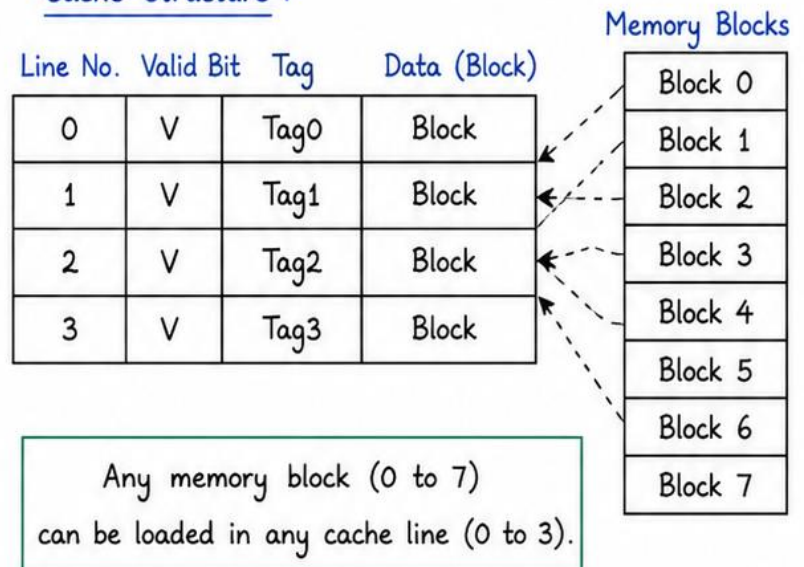
Assume main memory has 8 blocks (0 to 7) and cache has 4 lines.

- Block size = 1 word
- Number of lines in cache = 4
- Number of bits for block number = 3 (since $2^3 = 8$ blocks)
- Number of bits for line number = 0 (Fully Associative)
- Let word offset bits = 2 (for example)

Address format :

Tag (3 bits)			Word Offset (2 bits)	
4	3	2	1	0

Cache Structure :



6) Disadvantages :

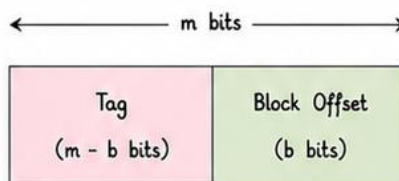
- Requires more hardware (comparators).
- Expensive.

Q.5] Explain Fully Associative Mapping with Example

FULLY ASSOCIATIVE MAPPING

- In Fully Associative Mapping, **any** block of main memory can be placed in **any** line of the cache.
- There is **no fixed location** for a block.
- To find a block, its Tag is compared with the Tag of all cache lines.
- If match is found → **Hit**
- If no match → **Miss** (block is loaded in any line using replacement policy).

ADDRESS FORMAT (Fully Associative)



Tag : Identifies the block.

Block Offset : Selects the word/byte in the block.

CACHE STRUCTURE

Line No.	Valid Bit	Tag	Data(Block)
0	V	Tag0	Block
1	V	Tag1	Block
2	V	Tag2	Block
3	V	Tag3	Block

Any memory block (0 to 7) can be loaded in any cache line (0 to 3).

Main Memory Blocks

Block 0
Block 1
Block 2
Block 3
Block 4
Block 5
Block 6
Block 7

EXAMPLE

Assume main memory has 8 blocks (0 to 7).

Cache has 4 lines.

Block size = 1 word.

- Number of lines in cache = 4
- Number of bits for block number = 3 (since $2^3 = 8$ blocks)
- Number of bits for line number = 0 (Fully Associative)
- Let word offset bits = 2 (for example)

Therefore :

Address bits = Tag bits (3) + Block Offset bits (2) = 5 bits

HOW IT WORKS?

- CPU generates an address and it is divided into Tag and Block Offset.
- The Tag is compared with the Tag of all cache lines simultaneously.
- If match is found → **Hit** (data found in cache).
- If no match → **Miss** (block is fetched from main memory and loaded in any line according to replacement policy).

Example of Mapping (Sequence of Accesses)

Access	Main Memory Block	Binary Block Number (3 bits)	Tag (3 bits)	Block Offset (2 bits)	Hit / Miss	Cache Content After Access			
						Line 0	Line 1	Line 2	Line 3
1	5	101	101	00	Miss	5	-	-	-
2	1	001	001	00	Miss	5	1	-	-
3	7	111	111	00	Miss	5	1	7	-
4	2	010	010	00	Miss	5	1	7	2
5	1	001	001	00	Hit	5	1	7	2
6	6	110	110	00	Miss	6 (replaced 5)	1	7	2

ADVANTAGES

- Least conflict misses.
- Most flexible mapping.
- Good performance.

DISADVANTAGES

- Requires more hardware (comparators for each line).
- More expensive.

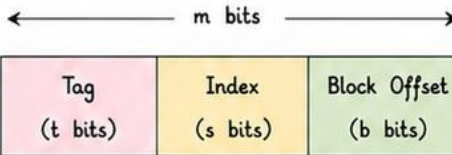
CONCLUSION : Fully associative mapping allows any block of main memory to be placed in any line of cache. It has high flexibility and least conflict misses but requires more hardware.

Q.3] Explain Set-Associative Mapping

SET-ASSOCIATIVE MAPPING

- Cache is divided into a number of sets and each set contains 'k' lines (k = associativity).
- A block of main memory can be placed in any line of its **corresponding set only**.
- It is a compromise between Direct Mapping and Fully Associative Mapping.

ADDRESS FORMAT (Set-Associative)



Tag : Identifies the block.

Index : Selects the set.

Block Offset : Selects the word/byte in the block.

HOW IT WORKS?

- Address is divided into Tag, Index and Block Offset.
- Index bits select the set in cache.
- Tag is compared with all lines in that set.
- If match is found → **Hit**.
- If no match → **Miss** (block is loaded in any line of that set using replacement policy).

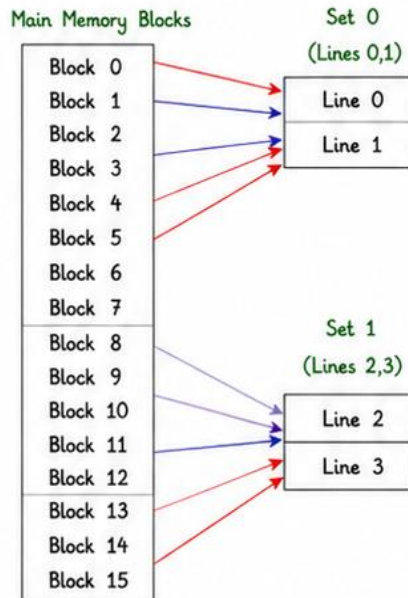
EXAMPLE

Assume main memory has 16 blocks (0 to 15).
Cache has 4 lines with 2-way set-associative mapping.
Block size = 1 word.

- Number of sets = $4/2 = 2$ sets
- Number of lines per set = 2 (2-way)
- Number of bits for block number = 4 (since $2^4 = 16$ blocks)
- Index bits (s) = $\log_2(2) = 1$ bit
- Block offset bits (b) = 0 (1 word block)
- Tag bits (t) = $4 - 1 - 0 = 3$ bits

Address bits = Tag (3) | Index (1) | Block Offset (0)

MAPPING DIAGRAM



CACHE STRUCTURE (2-WAY)

Set No.	Line No.	Valid Bit	Tag	Data (Block)
Set 0	Line 0	V	Tag	Block
	Line 1	V	Tag	Block
Set 1	Line 2	V	Tag	Block
	Line 3	V	Tag	Block

Each set has 2 lines (2-way).

A block can be placed in either line of its corresponding set.

ADVANTAGES

- Less conflict misses than Direct Mapping.
- Less hardware cost than Fully Associative Mapping.
- Good balance between performance and cost.

DISADVANTAGES

- More complex than Direct Mapping.
- Replacement policy required within each set.

CONCLUSION : Set-associative mapping divides cache into sets and allows a block to be placed in any line of its set only. It reduces conflict misses compared to direct mapping with less hardware than fully associative mapping.

Q.4] A cache has 64 blocks divided into 4 block sets and main memory has 4096 blocks, each consists of 128 words of 16-bit length. Find bits in main memory, tag, block, and word field.

Solution:

1. Total number of blocks in main memory

Main memory has 4096 blocks.

$$\begin{aligned}\text{Number of bits for block field} &= \log_2(4096) \\ &= 12 \text{ bits}\end{aligned}$$

2. Number of words in each block

Each block has 128 words.

$$\begin{aligned}\text{Number of bits for word field} &= \log_2(128) \\ &= 7 \text{ bits}\end{aligned}$$

3. Number of block sets in cache

Cache has 4 block sets.

$$\begin{aligned}\text{Number of bits for set (block) field} &= \log_2(4) \\ &= 2 \text{ bits}\end{aligned}$$

4. Number of block lines per set

Total blocks in cache = 64

$$\text{Number of lines per set} = 64 / 4 = 16$$

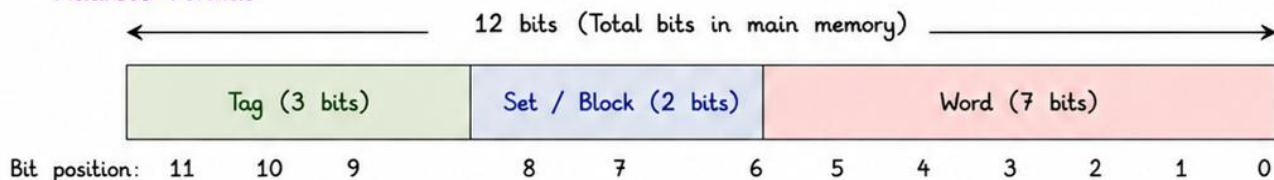
(This does not affect the address bits)

5. Calculate Tag bits

$$\begin{aligned}\text{Tag bits} &= \text{Total block bits} - (\text{Set bits} + \text{Word bits}) \\ &= 12 - (2 + 7) \\ &= 3 \text{ bits}\end{aligned}$$

Summary of Bits		
Field	Number of Bits	Explanation
Tag	3	Identifies the block
Set (Block)	2	Identifies the set (block set)
Word	7	Identifies the word in block
Total bits in main memory	12 bits	$3 + 2 + 7 = 12$

Address Format



Final Answer:

- Total bits in main memory address = 12 bits
- Tag field = 3 bits
- Block (Set) field = 2 bits
- Word field = 7 bits

$$12 = 3 \text{ (Tag)} + 2 \text{ (Set)} + 7 \text{ (Word)}$$

2.5 Replacement Algorithms

Q.3] Explain Replacement Algorithms (LRU, FIFO, LFU, Random)

Replacement algorithms are used in cache memory to **decide which block to remove** when a new block needs to be loaded and the cache is full.

1. LRU (Least Recently Used)

- Replaces the block that has not been used for the longest time in the past.
- It assumes that the block which was used recently is likely to be used again soon.
- Gives the minimum number of page (cache) misses.

Example:

Reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Cache size = 3

7	0	1	2	0	3	0	4	2	3	0	3
7	0	1	2	2	2	2	4	4	4	0	0
-	7	7	1	1	3	3	3	2	2	2	2
-	-	-	0	0	0	0	0	0	3	3	3

Page (Cache) Misses = 9

2. FIFO (First In First Out)

- Replaces the block that has been in the cache the longest time, regardless of how recently it was used.
- Implements a queue.
- Simple to implement but may cause more misses than LRU.

Example:

Reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Cache size = 3

7	0	1	2	0	3	0	4	2	3	0	3
7	0	1	2	2	2	0	0	2	3	0	0
-	7	7	0	0	3	3	3	3	3	3	3
-	-	-	1	1	1	1	4	4	4	3	4

Page (Cache) Misses = 10

3. LFU (Least Frequently Used)

- Replaces the block that has the lowest access frequency.
- If two blocks have the same frequency, LRU or FIFO is used to break the tie.
- Good for workloads with stable access patterns.

Example:

Reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Cache size = 3

7	0	1	2	0	3	0	4	2	3	0	3
7	0	1	2	2	3	3	4	2	3	0	3
-	7	7	0	2	0	3	0	2	3	3	3
-	-	-	1	1	1	2	?	0	0	0	0

Page (Cache) Misses = 9

4. Random (RAND)

- Replaces a block that is selected randomly from the cache.
- Very simple to implement.
- Performance is unpredictable.

Example:

Reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Cache size = 3

7	0	1	2	0	3	0	4	2	3	0	3
7	0	1	2	0	2	0	0	2	3	0	3
-	7	7	0	0	R	R	R	R	R	R	R
-	-	-	1	1	1	1	4	4	4	4	0

Page (Cache) Misses = Varies (depends on random choice)

Comparison

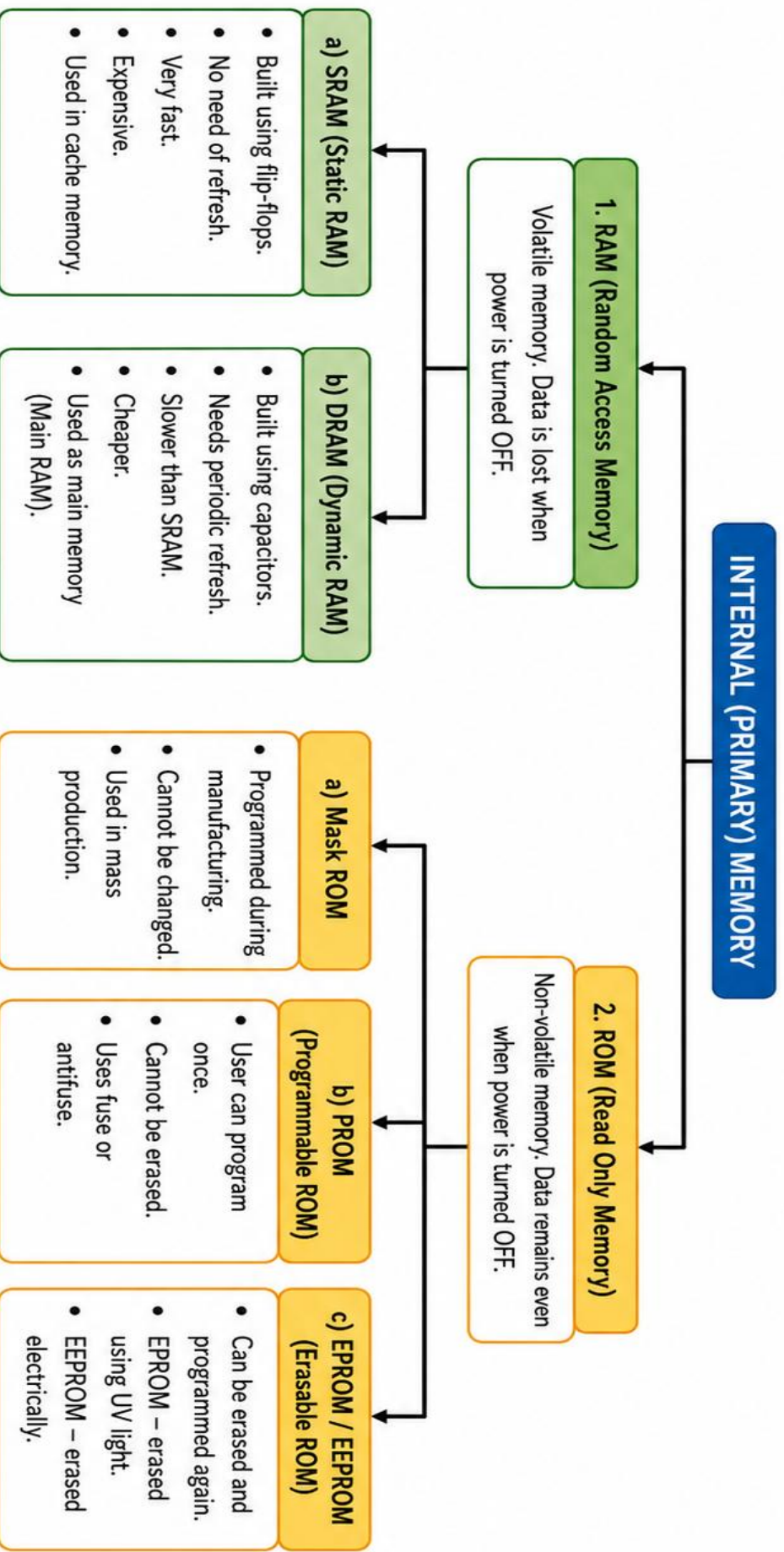
Algorithm	Policy	Advantages	Disadvantages	Performance
LRU	Least Recently Used	Minimum misses, good performance	Complex to implement (needs time info)	Best
FIFO	First In First Out	Simple to implement	May replace important blocks	Average
LFU	Least Frequently Used	Good for stable access patterns	Needs extra hardware (counters)	Good
Random	Randomly Selected	Very simple	Unpredictable, may cause more misses	Poor to Average

Conclusion: Replacement algorithms help in efficient use of cache and reduce cache misses.

LRU is the best but more complex, FIFO is simple, LFU is frequency based, and Random is simplest but unpredictable.

Q.4) Give detail Classification of Internal/Primary memory.

Internal (Primary) memory is the main memory that is directly accessed by the CPU. It stores data, instructions and results temporarily while the system is running.



Summary of Internal Memory

- RAM is used for temporary storage while programs are running.
- ROM is used to store permanent data / firmware.
- RAM is volatile, ROM is non-volatile.
- RAM is read & write memory, ROM is read only memory.

Memory Type	Volatile / Non-Volatile	Read / Write	Speed	Used For
SRAM	Volatile	Read & Write	Very Fast	Cache Memory
DRAM	Volatile	Read & Write	Slow (compared to SRAM)	Main Memory
ROM	Non-Volatile	Read Only	Slow	Firmware, BIOS

Q.2] Explain Advanced DRAM Organization

Advanced DRAM (Dynamic RAM) is a high density, high performance semiconductor memory in which each bit is stored in a capacitor and requires periodic refresh.

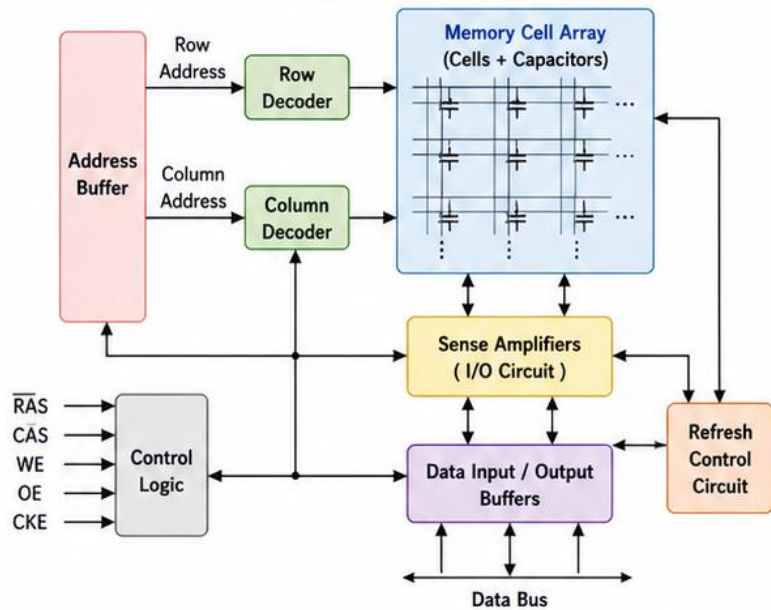
1. Introduction

DRAM is the main memory used in modern computer systems. It offers high storage capacity at low cost. Advanced DRAM uses improved cell structures, fast access techniques and internal architecture to achieve higher speed, higher density and lower power consumption.

3. Features of Advanced DRAM

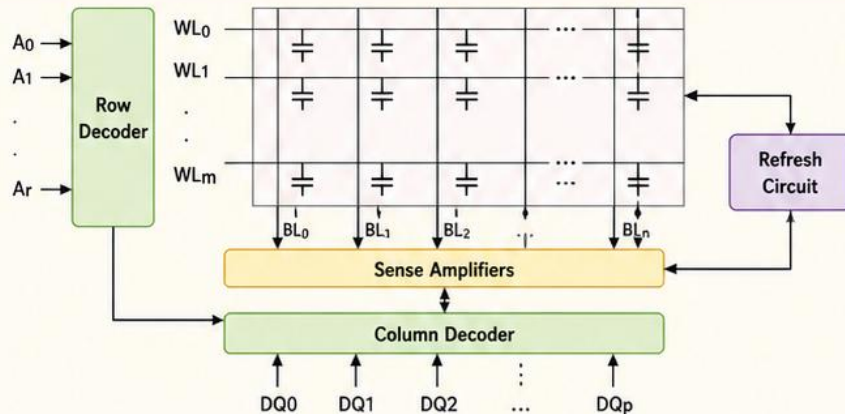
- **High Density:** More bits per chip using advanced cell design.
- **High Speed:** Faster access using pipelining, burst mode and prefetch.
- **Low Power:** Uses low power process and power down modes.
- **Reliability:** On-chip error detection and correction (ECC) in some DRAMs.
- **Refresh Management:** Efficient refresh to maintain data in capacitors.

2. Advanced DRAM – Block Diagram



4. Internal Organization of Advanced DRAM

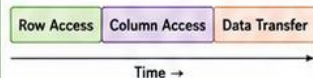
- **Memory Cell:** Each cell stores 1 bit of data in a capacitor and 1 transistor.
- **Cell Array:** Large number of cells arranged in rows and columns.
- **Row Decoder:** Selects one row (word line) at a time.
- **Column Decoder:** Selects one or more columns (bit lines).
- **Sense Amplifier:** Detects small charge on capacitor and restores full logic level.
- **Refresh Circuit:** Reads and writes back data periodically to maintain charge.



5. Advanced DRAM Techniques

(i) Pipelined Architecture

- Overlaps row access, column access and data transfer.
- Increases throughput and speed.



(ii) Burst Mode Operation

- Once a row is activated, multiple consecutive data words are transferred.
- Reduces overhead and improves performance.



(iii) Prefetch Architecture

- DRAM fetches multiple bits from memory in one access.
- Prefetch buffer holds data and supplies to CPU quickly.
- Common prefetch sizes: 2, 4, 8.



(iv) Low Power Techniques

- **Power Down Mode:** Disables unused banks.
- **Self Refresh Mode:** Refresh internally when system is idle.
- Helps in saving power in portable devices.

6. Summary

- Advanced DRAM organization uses a cell array with row and column decoders, sense amplifiers, I/O buffers and refresh circuitry.
- It provides high density, high speed and low power operation.
- Techniques like pipelining, burst mode and prefetch improve performance.
- Refresh operation is essential to maintain data in capacitors.

Q.3] Compare DRAM and SDRAM

DRAM (Dynamic RAM) is the basic dynamic memory.

SDRAM (Synchronous DRAM) is an advanced version of DRAM with synchronized operation.

Sr. No.	Parameter	DRAM (Dynamic RAM)	SDRAM (Synchronous DRAM)
1.	Full Form	Dynamic Random Access Memory	Synchronous Dynamic Random Access Memory
2.	Synchronization	Asynchronous. Operation is not synchronized with system clock.	Synchronous. All operations are synchronized with system clock.
3.	Operation	Data transfer is not synchronized with clock. Uses control signals for timing.	Data transfer occurs on the clock edge. Uses system clock for all operations.
4.	Speed	Slower than SDRAM.	Faster than DRAM.
5.	Bandwidth	Lower bandwidth. One operation at a time.	Higher bandwidth. Can perform pipelined and burst operations.
6.	Data Transfer	Single data transfer. Transfers one data word at a time.	Burst data transfer. Transfers multiple data words in one operation.
7.	Architecture	Simple internal architecture.	More complex architecture with pipeline and buffers.
8.	Clock Usage	Does not use system clock.	Uses system clock for timing and synchronization.
9.	Performance	Lower performance.	Higher performance.
10.	Cost	Cheaper.	Costlier than DRAM.
11.	Voltage	Operates at higher voltage (5V).	Operates at lower voltage (3.3V or 2.5V).
12.	Power Consumption	Higher power consumption.	Lower power consumption.
13.	Use	Used in older memory systems.	Used in modern computer systems.
14.	Example	FPM DRAM, EDO DRAM.	SDRAM, DDR SDRAM, DDR2/3/4 SDRAM.

Summary :

SDRAM is an advanced version of DRAM. It is synchronized with clock, provides higher speed, higher bandwidth and better performance through burst and pipelined operations. Hence, SDRAM is widely used in modern systems.

Q.2] Explain Hard Disk Organization with diagram

Introduction :

A Hard Disk is a non-volatile secondary storage device that stores large amount of data permanently. It uses magnetic recording to store data on rotating platters (disks).

1. Main Components of Hard Disk

1. Platters :

It consists of one or more circular disks made of aluminum or glass coated with magnetic material.

2. Spindle :

All platters are mounted on spindle which rotates at high speed (e.g., 5400 or 7200 RPM).

3. Read/Write Heads :

Each surface of platter has a read/write head which reads or writes data magnetically.

4. Actuator Arm :

It holds the read/write heads and moves inwards or outwards across the platter surfaces.

5. Actuator Axis (Pivot) :

It is the point around which the actuator arm moves.

6. Controller (PCB) :

It controls all operations like head movement, data transfer, error checking, etc.

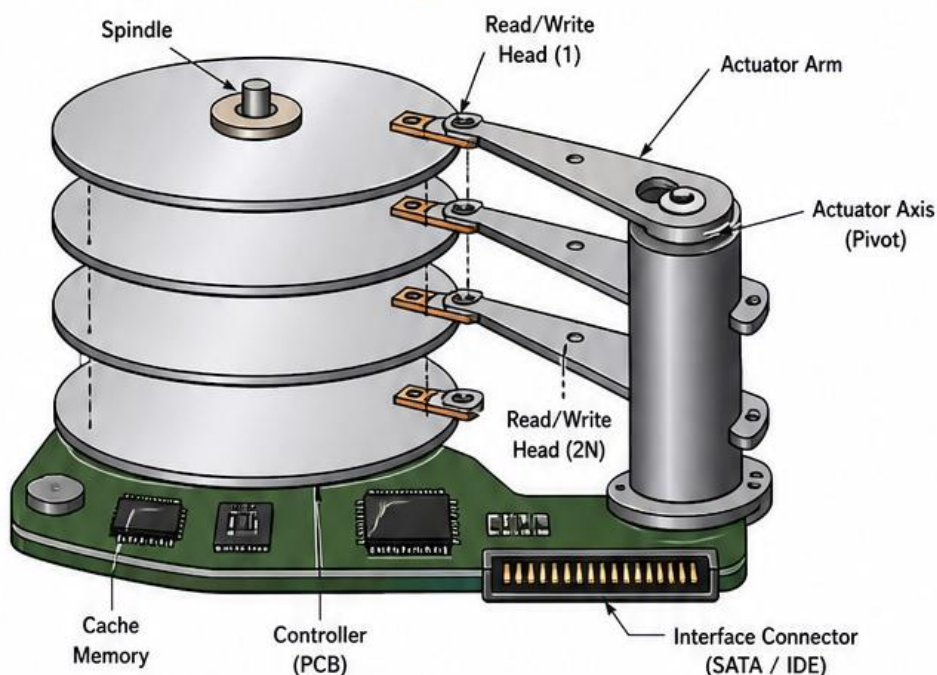
7. Interface Connector :

It connects the hard disk to the computer system (SATA / IDE).

8. Cache Memory :

It is a small high-speed memory used to store frequently accessed data.

2. Hard Disk Drive – Block Diagram



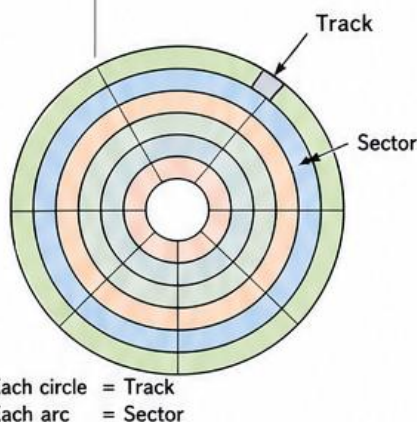
3. Organization of Data on Hard Disk

Data is stored on the platter surfaces in concentric tracks.

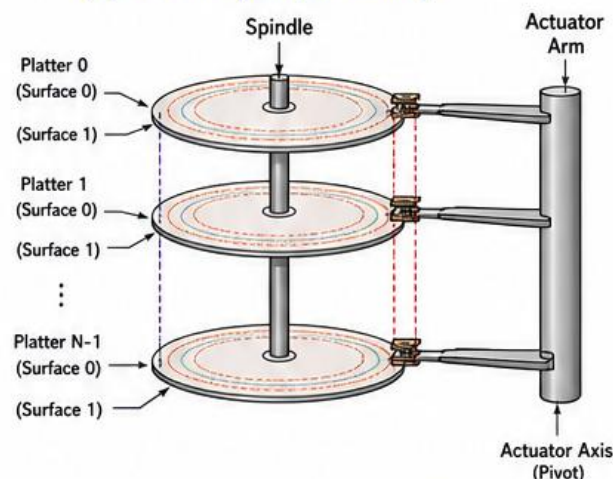
Each track is divided into sectors.

The smallest unit of data transfer is one sector (usually 512 Bytes or 4 KB).

(a) Top View of Platter



(b) Side View (Multiple Platters)



4. Working

- The platter rotates at constant speed.
- The actuator arm positions the head on the required track.
- The head reads data from or writes data to the sectors on that track.
- Data is transferred between hard disk and computer through controller and interface.

5. Key Points

Parameter	Description
Tracks	Concentric circles on a platter surface.
Sectors	Division of track; smallest data unit.
Cylinder	Set of tracks at same radius on all platters.
Access Time	Time to position head over data.
Rotational Latency	Time for required sector to rotate under the head.

Q.4] Evaluate disk based on disk performance parameter

Disk performance depends on the time taken to read/write data.

It is evaluated using the following main parameters.

Sr. No.	Parameter	Description	Depends on	Effect on Performance
1.	Access Time	Time to position the read/write head over the required track.	- Seek time of actuator arm - Distance of seek (number of tracks)	Lower access time → Better performance
2.	Rotational Latency	Time waiting for the required sector to rotate under the head.	Rotational speed (RPM)	Lower rotational latency → Better performance
3.	Transfer Time	Time actually required to transfer data once the sector is under the head.	- Disk rotational speed - Number of bytes per sector	Lower transfer time → Better performance
4.	Data Transfer Rate	Rate at which data is transferred between disk and system.	- Disk speed - Interface (SATA/IDE etc.)	Higher transfer rate → Better performance
5.	Disk Throughput	Total amount of user data transferred per unit time.	All above parameters combined	Higher throughput → Better performance

How parameters affect performance?

- Access Time (Seek Time):**
If the head has to move more tracks, access time increases.
Less access time means faster response.  Minimize seek time for better performance.
- Rotational Latency:**
Average rotational latency = 1/2 of one rotation.  Higher RPM (e.g., 7200 RPM) gives less latency and better performance.
- Transfer Time:**
More data per sector or higher RPM reduces transfer time.  Faster disk and larger sectors improve transfer time.
- Data Transfer Rate:**
Depends on disk speed, interface (e.g., SATA 3, SAS) and cache size.  Higher data transfer rate improves overall speed.
- Disk Throughput:**
Depends on access time, latency, transfer time and transfer rate.  Optimizing all parameters increases throughput.

Performance Evaluation (Formula)

$$\text{Average Access Time} = \text{Access Time} + \text{Rotational Latency} + \text{Transfer Time}$$
$$\text{Throughput} = \frac{\text{Total Data Transferred}}{\text{Total Time Taken}}$$

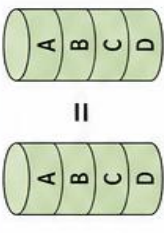
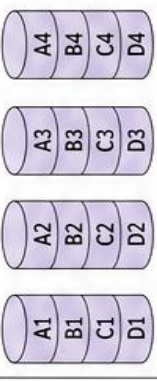
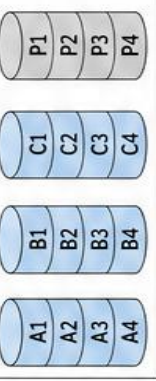
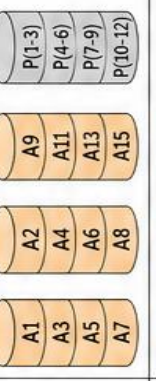
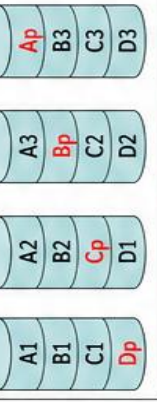
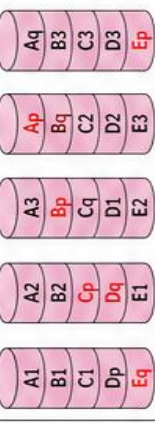
Where,

- Access Time = time to move head to track
- Rotational Latency = time for sector to rotate under head
- Transfer Time = time to transfer data once sector is under head

Summary: A good disk has less access time, less rotational latency, less transfer time, higher data transfer rate and higher throughput. These parameters together decide the performance of a disk.

Q.5] What is RAID levels 1 to 6 – Advantages and Disadvantages

RAID (Redundant Array of Independent Disks) is a technology that combines multiple hard disks to improve **performance**, increase **reliability**, and provide **data redundancy**.

RAID Level	Name (Alternate Name)	How it Works	Advantages	Disadvantages	Diagram (Example)
RAID 1	Mirroring	Data is exactly mirrored on two or more disks. If one disk fails, other disk has the same data.	✓ High reliability ✓ Very simple ✓ Fast read performance	✗ 50% storage efficiency (used = total / 2) ✗ Cost is high	 2 Disks (Example)
RAID 2	Bit-level Striping with Hamming Code	Data is striped at bit level. Uses Hamming Code for error correction.	✓ High data protection ✓ Error correction possible	✗ Very complex ✗ Expensive ✗ Rarely used now	 4 Disks (Example)
RAID 3	Byte-level Striping with Parity	Data is striped at byte level with dedicated parity disk.	✓ Good for large sequential data ✓ Parity gives protection	✗ Parity disk can be bottleneck ✗ Not good for small files	 4 Disks (Example)
RAID 4	Block-level Striping with Parity	Data is striped at block level with dedicated parity disk.	✓ Good read performance ✓ Data protection with parity	✗ Parity disk is bottleneck ✗ Not suitable for heavy write operations	 4 Disks (Example)
RAID 5	Block-level Striping with Distributed Parity	Data is striped at block level and parity is distributed across all disks.	✓ No dedicated parity disk (better performance) ✓ Good read & write performance ✓ Efficient storage use	✗ Rebuilding after disk failure takes time ✗ Write performance lower than RAID 1	 4 Disks (Example)
RAID 6	Block-level Striping with Dual Parity	Similar to RAID 5 but uses two parity blocks for each set of data.	✓ Very high data protection (can survive 2 disk failures) ✓ Good for large systems	✗ More parity overhead (less storage efficiency) ✗ Write performance lower	 5 Disks (Example)

Note:

- p = Parity
- At least minimum number of disks are required as shown in diagrams.
- RAID 0 is not included here as it provides only striping (no redundancy).

Quick Summary

- RAID 1 – Best reliability, poor storage use.
- RAID 2 – Not used now.
- RAID 3 & 4 – Have dedicated parity disk (bottleneck).
- RAID 5 – Best balance of performance & storage.
- RAID 6 – Higher protection (tolerates 2 failures).